

Intermediate (test a tool without calling the LLM)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `tool_test_no_llm_practice.py`: an agent bug is usually either "the tool did the wrong thing" or "the model called the wrong tool". Testing the tool alone, with no model, settles the first question for free, every time. **If not:** every tool bug would show up only through the full agent, mixed up with the model's own choices, and cost an API call each time you checked.

Copy Doc06's `tools.py` into `practice/` first, with the files it imports (`http_client.py` , `exceptions.py` , `logging_setup.py`), unchanged. Run every version from inside `practice/` with `pytest tool_test_no_llm_practice.py -v` . Read both depths — they're not "wrong, right," they're 2 real, valid ways to test the same tools, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/tool_test_no_llm_practice.py
from tools import add

def test_add():
    result = add.invoke({"a": 2, "b": 3})
    assert result == 5
```

Expected output:

```
tool_test_no_llm_practice.py::test_add PASSED
```

This is a correct, minimal test: no model, no API key, no network — just the tool. `add` is a LangChain `@tool` , so it's called with `.invoke()` and a dict, the same way Doc06's `tool_harness.py` ran it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the success path and the failure path of each tool

Story: a model will sometimes call a tool that fails. Doc06's `flaky_lookup` catches its own error and returns an `"Error: ..."` string, so the agent can read it and recover. A test on that exact string locks the behavior in. **If not:** someone could "tidy up" the tool so it raises instead, and the agent would start crashing mid-run with no test to warn you.

```
# practice/tool_test_no_llm_practice.py
from tools import add, flaky_lookup

def test_add_returns_the_sum():
    # why: pure math – exact == is the right check
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    # how: should_fail=True forces the failure on purpose – no
    # waiting for a real outage to see what the tool does
    result = flaky_lookup.invoke(
        {"query": "refund policy", "should_fail": True}
    )
    # why: the tool must hand back a readable error, not crash
    assert result == "Error: simulated failure"
```

Expected output:

```
tool_test_no_llm_practice.py::test_add_returns_the_sum PASSED
tool_test_no_llm_practice.py::test_flaky_lookup_success PASSED
tool_test_no_llm_practice.py::test_flaky_lookup_failure PASSED
```

Approach 2 — wrong argument types are rejected before the tool runs

Story: a model sometimes sends `"two"` where a number belongs. `@tool` checks the arguments against the type hints before the function runs, and `pytest.raises` proves it. **If not:** you'd only be hoping that bad arguments get stopped at the door, instead of knowing it.

```
# practice/tool_test_no_llm_practice.py
import pytest
from pydantic import ValidationError
from tools import add

def test_add_rejects_wrong_type():
    # how: the test passes only if the block inside raises
    # ValidationError – and fails if nothing is raised
    with pytest.raises(ValidationError):
        add.invoke({"a": "two", "b": 3})

def test_add_rejects_missing_argument():
    # when: the model leaves out a required argument
    with pytest.raises(ValidationError):
        add.invoke({"a": 2})
```

Expected output:

```
tool_test_no_llm_practice.py::test_add_rejects_wrong_type PASSED
tool_test_no_llm_practice.py::test_add_rejects_missing_argument PASSED
```

`get_weather` isn't tested in this file on purpose: it calls a real weather API, so a test for it would need the network — which is exactly what this exercise keeps out.

Difference from Basic: Approach 1 tests both the success and the failure path of each tool, with exact checks, so the tool's "return an error string, don't crash" behavior can't quietly change. Approach 2 checks what happens *before* the tool runs: wrongly typed or missing arguments are rejected with a `ValidationError`, proven with `pytest.raises`.

Which one should you actually write? Approach 1 for every tool you write — a success test and a failure test each. Add Approach 2's checks for tools whose argument types really matter (numbers, IDs, dates), since that's where a model's bad arguments do the most damage. The Build Task's `test_unit_layer.py` reuses Approach 1.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Revision Add-on — the same no-model testing, pointed at Doc01's config loader

Story: Doc01's `load_config()` is supposed to fail loudly when `OPENAI_API_KEY` is missing — the kind of rule nobody tests until a deploy breaks. pytest's built-in `monkeypatch` changes environment variables and functions only for one test, then puts everything back. **If not:** a later "cleanup" of `config.py` could make a missing key return `None` silently, and you'd find out in production.

Copy Doc01's `config.py` into `practice/` too. Your `exceptions.py` needs Doc01's `MissingConfigError` as well as Doc02's two HTTP errors — keep all three classes in that one file.

```
# practice/tool_test_no_llm_practice.py – Revision section
import pytest
import config                                # your Doc01 config.py
from exceptions import MissingConfigError     # Doc01's error class

def do_not_load_dotenv():
    # why: stands in for load_dotenv() during the test, so your real
    # .env file can't quietly put the key back
    return None

def test_missing_api_key_raises(monkeypatch):
    # how: removes the variable for this test only
    monkeypatch.delenv("OPENAI_API_KEY", raising=False)
    # how: swaps config.load_dotenv for the fake above, this test only
    monkeypatch.setattr(config, "load_dotenv", do_not_load_dotenv)

    with pytest.raises(MissingConfigError):
        config.load_config()

def test_log_level_defaults_to_info(monkeypatch):
    monkeypatch.setenv("OPENAI_API_KEY", "sk-test-123")
    monkeypatch.delenv("LOG_LEVEL", raising=False)
    monkeypatch.setattr(config, "load_dotenv", do_not_load_dotenv)

    assert config.load_config().log_level == "INFO"
```

Expected output:

```
tool_test_no_llm_practice.py::test_missing_api_key_raises PASSED  
tool_test_no_llm_practice.py::test_log_level_defaults_to_info PASSED
```

`monkeypatch` undoes every change when each test ends, so your real environment is never touched. Two tiny tests lock Doc01's "fail loudly" and "sensible default" rules in place, so a later refactor can't quietly remove them.